POLITECNICO
MILANO 1863

System Dependability: Part 2

Reliability, Availability, and Reliability Block Diagrams

Roberto Sala

March 16, 2026

Today's Agenda

1. Reliability
2. Availability
3. Key Indices
4. Real-World Incidents
5. Reliability Block Diagrams
6. Advanced Redundancy
7. Design Examples
8. Summary

Reliability: The Formal Definition

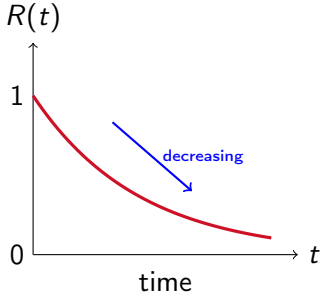
Definition

The ability of a system or component to perform its required functions under stated conditions for a **specified period of time**.

Mathematical definition:

$R(t) = P(\text{not failed during } [0, t] \mid \text{working at } t = 0)$

- $R(0) = 1$ (starts working)
- $R(t \to \infty) = 0$ (eventually fails)
- $R(t)$ is non-increasing
- $Q(t) = 1 - R(t)$ (unreliability)



Time Horizon Matters

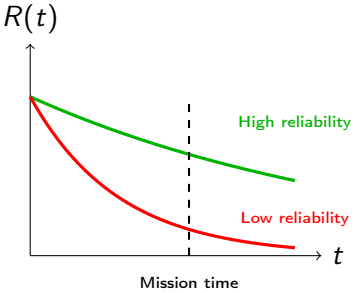
Mission-Based Reliability

"If a system needs to work for slots of ten hours at a time, then **ten hours is the reliability target**."

Use cases:

- Safety-critical systems with no repair
- Spacecraft, submarines, aircraft flights
- Industrial processes

Key insight:
Even small periods of incorrect behavior are unacceptable in these systems.



Exponential Distribution (Most Common Case)

For constant failure rate λ:

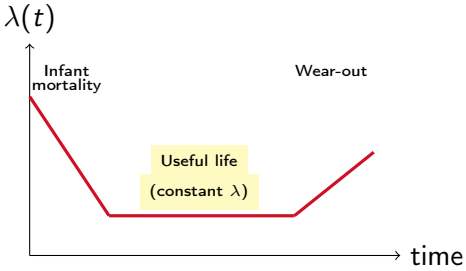
$$R(t) = e^{-\lambda t}$$

During the **useful life period**, systems exhibit approximately constant failure rates.

Properties:

- Memoryless: future failures independent of past
- Mathematically tractable
- Good approximation for electronic systems

Bathtub Curve



We model the useful life period

Availability: Readiness for Service

Definition

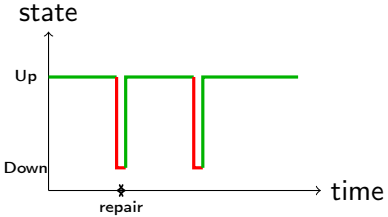
The degree to which a system or component is **operational and accessible** when required for use.

Mathematical definition:

$$A(t) = P(\text{operational at time } t)$$

Key differences from reliability:

- Admits **brief outages**
- Instant in time, not interval
- Can be restored after failure



Availability: Readiness for Service

Definition

The degree to which a system or component is **operational and accessible** when required for use.

Special cases:

- Non-repairable: $A(t) = R(t)$
- Repairable: $A(t) \geq R(t)$

$$A = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}}$$

The "Nines" of Availability

Nines	Availability	Downtime/year	Example
1	90%	5 weeks	Unacceptable
2	99%	4 days	Generic website
3	99.9%	9 hours	Amazon.com
4	99.99%	1 hour	Enterprise server
5	99.999%	5 minutes	Telephone system
6	99.9999%	30 seconds	Network switches
7	99.99999%	3 seconds	Theoretical limit

Reality Check:

Each additional "nine" becomes **exponentially harder and more expensive** to achieve!

Reliability vs Availability: The Key Difference

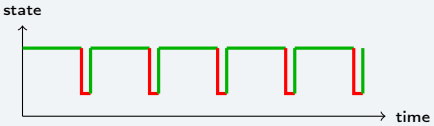
Reliability = continuous service | **Availability = ready when needed**

High Reliability

Long uninterrupted uptime

Continuous correct service

High Availability



Frequent brief outages

Fast recovery

Reliability vs Availability: The Key Difference

Reliability = continuous service | **Availability = ready when needed**

Example

A system that fails once per hour but recovers in 10ms:

- **Low reliability** (60 failures per day!)
- **High availability** ($A = 3600/(3600 + 0.01) = 99.9972\%$)

MTTF and MTBF

Mean Time To Failure (MTTF)

Mean time before **first failure**

$$\text{MTTF} = \int_0^\infty R(t)\,dt$$

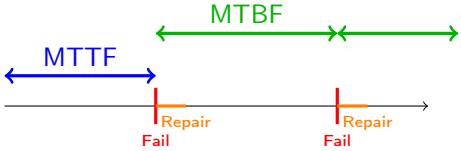
For exponential: $\text{MTTF} = 1/\lambda$

Mean Time Between Failures (MTBF)

Mean time between **consecutive failures**

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

Timeline Illustration



Key Assumption

If repair time negligible:
$\text{MTBF} \approx \text{MTTF}$

MTTR and MDT

Mean Time To Repair (MTTR)

Time to detect + repair + recover

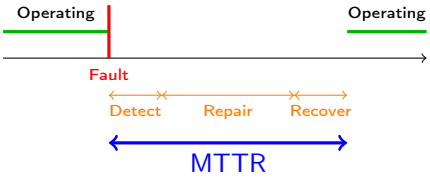
$$\text{MTTR} = t_{\text{detect}} + t_{\text{repair}} + t_{\text{recover}}$$

Mean Down Time (MDT)

Time system is not working
May differ from MTTR if:

- Failure not noticed immediately
- Repair delayed intentionally
- System not restarted immediately

Detailed Timeline



Critical Note

Detection overhead can be significant!

Availability Formula

For Repairable Systems

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}}$$

Example

A server has:

- MTTF = 8760 hours (1 year)
- MTTR = 8.76 hours (approximately 0.1% of MTTF)

Calculate availability.

Availability Formula

Example

$$A = \frac{8760}{8760 + 8.76} = \frac{8760}{8768.76} = 0.999 = 99.9\%$$

This is **three nines** availability (about 9 hours downtime per year).

Design Insight

To improve availability: increase MTTF **or** decrease MTTR (or both!)

FIT: Failures In Time

Definition

FIT = Expected failures per **billion hours** (10^9 hours)

$$\text{FIT} = \frac{10^9}{\text{MTBF (hours)}}$$

$$\text{MTBF (hours)} = \frac{10^9}{\text{FIT}}$$

Why use FIT?

- Common in hardware datasheets
- Easier to work with small failure rates
- Industry standard metric

FIT: Failures In Time

Example

A component with 100 FIT:

$$\text{MTBF} = \frac{10^9}{100} = 10^7 \text{ hours}$$

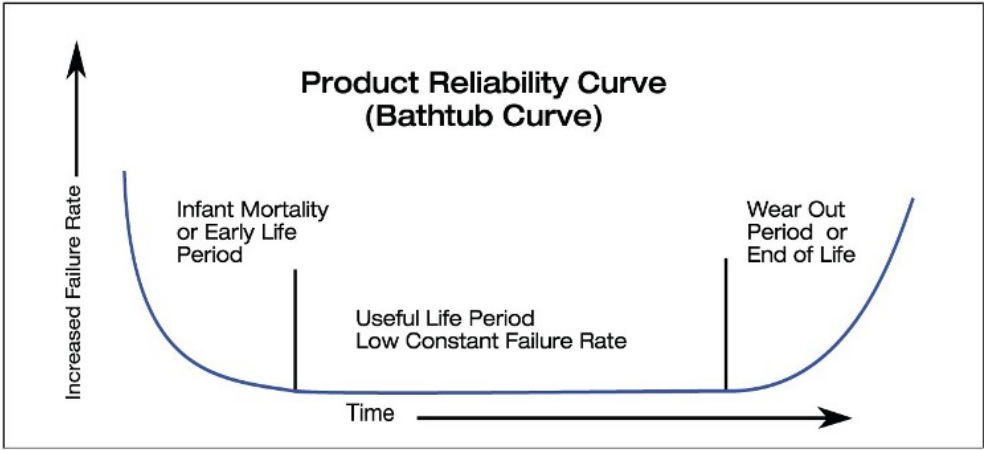
That's approximately:

- 1,141 years!
- Very reliable component

Perspective

Even "reliable" components fail. With thousands of components, system reliability becomes critical.

Failure Rate and the Bathtub Curve



Failure Rate and the Bathtub Curve

Infant Mortality

- Defective components
- Manufacturing issues
- **Solution:** Burn-in testing

Useful Life

- Constant failure rate
- Random failures
- **Our focus:** Model as exponential

Wear-Out

- Aging components
- Increased failures
- **Solution:** Preventive maintenance

AT&T Outage – February 22, 2024

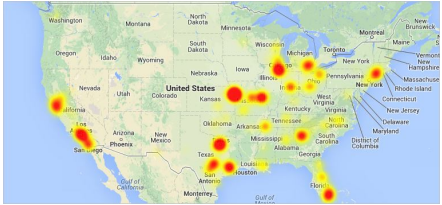
Incident Details (*More details here.*)

Cause: Network expansion with incorrect process/configuration

Impact:

- All 50 US states + territories
- 125+ million devices affected
- 92+ million calls blocked
- 25,000+ failed 911 calls
- FirstNet (public safety) down

Duration: At least 12 hours to full restoration



Metrics

Availability violated

MTTR = 12 hours

(Very poor for telecom)

If annual: $A \approx 99.86\%$

Google Cloud Outage – June 12, 2025

Incident Details (*More details here.*)

Cause: Malformed quota policy in Service Control system

Impact: 3-hour global outage

- Compute Engine, Cloud Storage, BigQuery affected
- Gmail disrupted
- API errors, 503 responses

Recovery: Manual rollback, region-by-region stabilization

Metrics:

Detection: 4 minutes (good!)
MTTR: 3 hours

If annual: $A \approx 99.97\%$

Lesson learned:

Fast detection $\neq$ fast recovery in distributed systems

Lessons from Real Incidents

Common Patterns

1. **Software/configuration errors**, not hardware failures
2. **Change management** failures (updates, rollouts)
3. **Manual recovery** required (high MTTR)
4. **Cascading effects** in distributed systems
5. **Single points of failure** in critical paths

Lessons from Real Incidents

Comparison

Which had better availability?

- AT&T: 12-hour MTTR
- Google: 3-hour MTTR

Google's faster recovery shows better **maintainability**.

Design Question

How to prevent these?

- Staged rollouts
- Canary deployments
- Automated rollback
- Redundant control planes

What Are RBDs?

Definition

An **inductive model** where a system is divided into blocks representing distinct elements (components or subsystems).

Key properties:

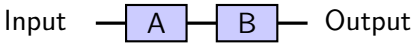
- Each block has its own reliability $R_i(t)$
- Blocks combined to model **success paths**
- System works if **at least one path** from input to output works
- **RBD topology** $\neq$ **physical topology**

Purpose:
Compute system reliability from component reliabilities

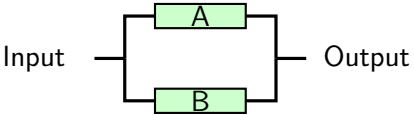
What Are RBDs?

Definition

An **inductive model** where a system is divided into blocks representing distinct elements (components or subsystems).



Series: both must work



Parallel: at least one must work

Series Systems

Series Configuration

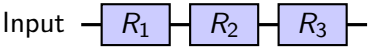
All components must work for system to work

$$R_S(t) = \prod_{i=1}^{n} R_i(t)$$

For exponential distributions:

$$R_S(t) = e^{-\lambda_S t}, \qquad \text{MTTF}_S = \frac{1}{\lambda_S}$$

where $\lambda_S = \sum_{i=1}^{n} \lambda_i$



Key Insight

Series system is **less reliable** than its least reliable component!

Example

Three components, $R = 0.9$ each:

$$R_S = 0.9^3 = 0.729$$

Parallel Systems

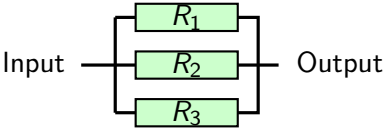
Parallel Configuration

At least one component must work for system to work

$$R_S(t) = 1 - \prod_{i=1}^{n}(1 - R_i(t))$$

For identical components:

$$R_S(t) = 1 - (1 - R)^n$$



Key Insight:

Parallel system is **more reliable** than its most reliable component!

Example

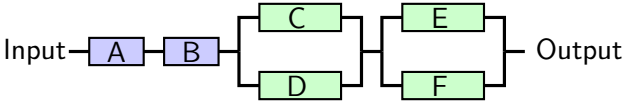
Two components, $R = 0.9$ each:

$$R_S = 1 - (1 - 0.9)^2 = 0.99$$

Worked Example: Complex System

Problem: Calculate system reliability given component reliabilities:

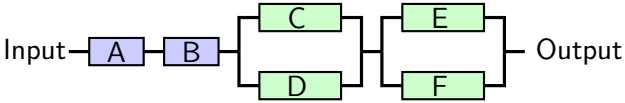
- $R_A = 0.95$, $R_B = 0.97$, $R_C = 0.99$, $R_D = 0.99$, $R_E = 0.92$, $R_F = 0.92$



Worked Example: Complex System

Problem: Calculate system reliability given component reliabilities:

- $R_A = 0.95$, $R_B = 0.97$, $R_C = 0.99$, $R_D = 0.99$, $R_E = 0.92$, $R_F = 0.92$



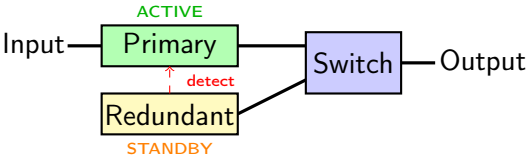
Solution:

1. $R_{AB} = R_A \times R_B = 0.95 \times 0.97 = 0.9215$
2. $R_{CD} = 1 - (1 - R_C)(1 - R_D) = 1 - (0.01)(0.01) = 0.9999$
3. $R_{EF} = 1 - (1 - R_E)(1 - R_F) = 1 - (0.08)(0.08) = 0.9936$
4. $R_S = R_{AB} \times R_{CD} \times R_{EF} = 0.9215 \times 0.9999 \times 0.9936 = 0.9155$

Standby Redundancy

Concept

- **Primary** component active
- **Backup** component off (or idle)
- Backup activated on primary failure



Requirements:

- Fault detection mechanism
- Switching mechanism

For perfect switching, equal rates:

$$R(t) = e^{-\lambda t}(1 + \lambda t), \quad \text{MTTF} = \frac{2}{\lambda}$$

Advantage:

Backup component not aging during standby!

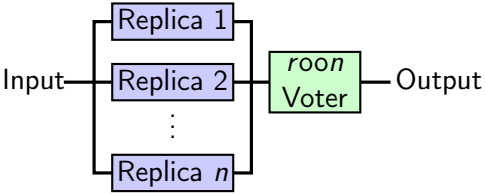
Risk:

Switching mechanism is a single point of failure!

r-out-of-n Redundancy

Concept

System has n components.
System works if **at least** r work.



Formula (with voter):

$$R_S = R_v \sum_{i=r}^{n} \binom{n}{i} R^i (1-R)^{n-i}$$

where R_v is voter reliability.

Key Point

Voter must be **highly reliable** — it's a single point of failure!

r-out-of-n Redundancy

Concept

System has n components.
System works if **at least** r work.

Formula (with voter):

$$R_S = R_v \sum_{i=r}^{n} \binom{n}{i} R^i (1-R)^{n-i}$$

where R_v is voter reliability.

Examples:

- 1oo2: Parallel (at least 1 of 2)
- 2oo3: TMR (at least 2 of 3)
- 3oo5: Majority of 5

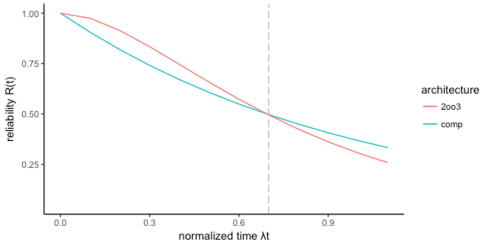
Triple Modular Redundancy (TMR)

2-out-of-3 with Voter

$$R_{TMR}(t) = R_v[3R^2 - 2R^3]$$

where:

- R = component reliability
- R_v = voter reliability



- $\text{MTTF}_{TMR} < \text{MTTF}_{simplex}$
- But $R_{TMR}(t) > R_{simplex}(t)$ for **short missions**
- Crossover at $t \approx 0.7 \times \text{MTTF}$

When to use TMR:

- Mission time < 0.7 MTTF
- Transient fault tolerance
- Cannot afford single failure

Triple Modular Redundancy (TMR)

2-out-of-3 with Voter

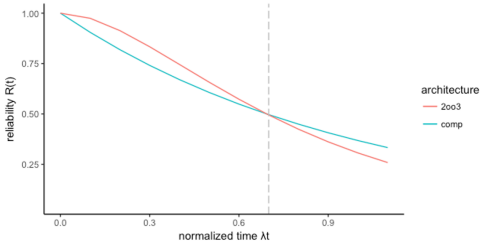
$$R_{TMR}(t) = R_v[3R^2 - 2R^3]$$

where:

- R = component reliability
- R_v = voter reliability

Paradox:

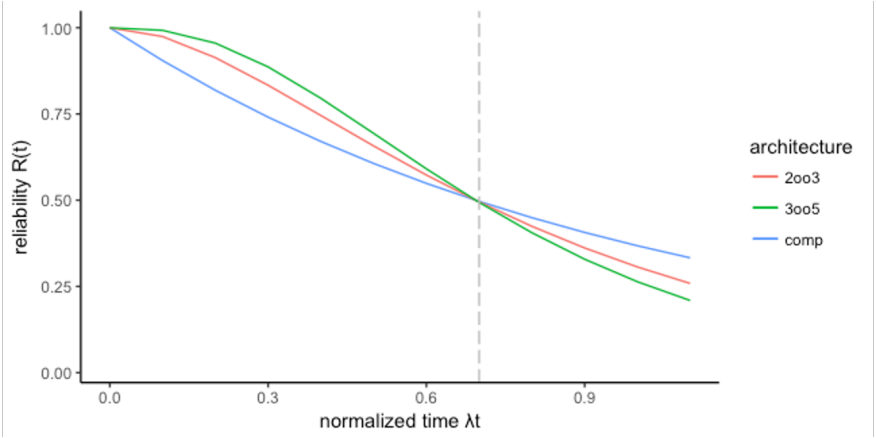
TMR adds components, so more can fail!
But for short missions, redundancy wins.



When NOT to use TMR:

- Long-term operation
- Cost-sensitive
- Voter is unreliable

Comparing Redundancy: 2oo3 vs 3oo5



Comparing Redundancy: 2oo3 vs 3oo5

Observations

- Higher $n \rightarrow$ better short-term reliability
- All redundant systems worse long-term
- Crossover points differ

Design Lesson:

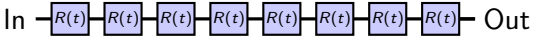
Redundancy is useful for specific mission times.

Choose n and r based on:

- Expected mission duration
- Component reliability
- Cost constraints

RAID-0 Example

Problem: 8-disk RAID-0 (striping) system. Each disk has reliability $R(t)$. What is system reliability?



Answer: Series configuration!
Any disk failure = total data loss

$$R_{RAID0}(t) = R(t)^8$$

If $R(t) = 0.99$: $R_{RAID0} = 0.99^8 = 0.923$

Key Lesson:

RAID-0 has NO redundancy!
Reliability **decreases** with more disks.

Contrast

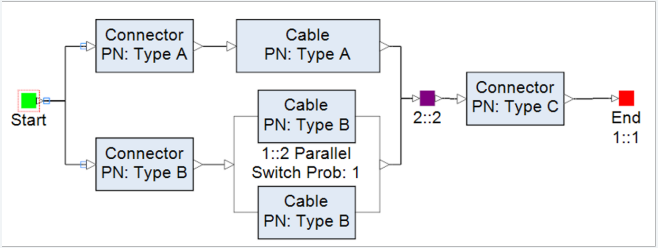
RAID-1 (mirroring):

- 4 mirrored pairs
- Each pair: $1 - (1 - R)^2$
- Much higher reliability

Design Trade-offs: Cable Bundle Example

Scenario: 6-component system, each $R = 0.9$, cost=1

Baseline

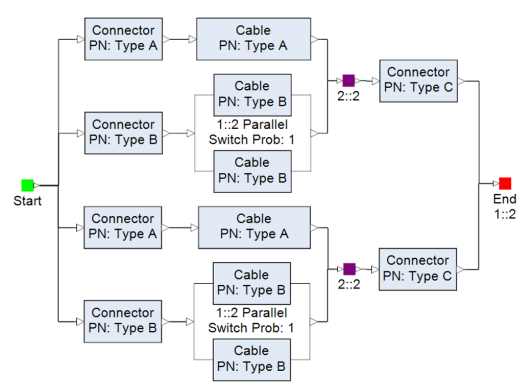


All in series: $R = 0.9^6 = 0.6495$, Cost $= 6$

Low reliability

Design Trade-offs: Cable Bundle Example

Alternative 1



Full duplication (system redundancy)

$$R = 1 - (1 - 0.6495)^2$$

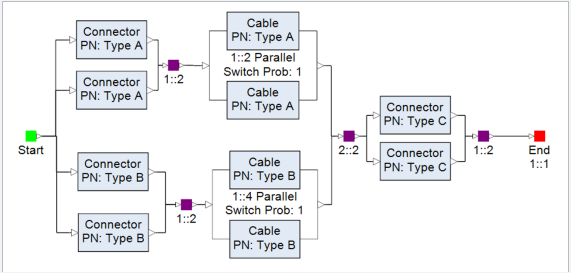
$$R = 0.8772$$

Cost = 12

Better, but expensive

Design Trade-offs: Cable Bundle Example

Alternative 2



Selective duplication
(component redundancy)

$$R = 0.9509$$

Cost = 10

Best trade-off!

Design Lesson:

Smart redundancy (component-level) beats brute-force (system-level).
Identify critical components and duplicate only those.

Finding the Right Trade-off

Design space exploration:

Application	Reliability Target	Strategy	Cost
Safety-critical	99.999%+	r-out-of-n, diverse	Very high
Mission-critical	99.99%	Standby, fast switching	High
Business-critical	99.9%	Parallel, load balancing	Moderate
Consumer	95–99%	Minimal redundancy	Low

Finding the Right Trade-off

Key Questions

1. What is your reliability target?
2. What is your cost budget?
3. What is acceptable downtime?
4. Can you tolerate performance overhead?
5. What are critical components?

Reality:

"What is good enough for your application?"

Perfect reliability is:

- Theoretically impossible
- Practically unaffordable
- Often unnecessary

Balance: dependability vs cost vs performance

Key Takeaways from Today

1. **Reliability vs Availability**
   - Reliability: $R(t) = e^{-\lambda t}$ (continuous service)
   - Availability: $A = \text{MTTF}/(\text{MTTF} + \text{MTTR})$ (readiness)
2. **Key Indices: MTTF, MTBF, MTTR, FIT**
   - Measure and predict system behavior
   - Design for high MTTF or low MTTR (or both!)

Key Takeaways from Today

3. **RBDs: Systematic Reliability Computation**
   - Series: $R_S = \prod R_i$ (reliability decreases)
   - Parallel: $R_S = 1 - \prod(1 - R_i)$ (reliability increases)
4. **Redundancy Strategies**
   - Standby, r-out-of-n, TMR
   - Useful for specific mission times
   - Component redundancy > system redundancy
5. **Real Incidents: Configuration errors, long MTTR, cascading failures**

Next Steps

Homework Exercise

Problem: Design a data storage system with 12 disks, each $R = 0.95$.

Compare:

- (A) RAID-0 (striping)
- (B) RAID-1 (mirroring pairs)
- (C) RAID-5 (single parity)

Questions:

1. Which achieves 99.9% reliability?
2. What is the cost in usable capacity?
3. Which has best cost-reliability trade-off?

Questions?

Roberto Sala
roberto.sala@polimi.it

Office: DEIB, 3rd floor, Building 22, Office 004
Send email to schedule appointment